Домашнее задание 14 — Введение в проект Loom (Virtual Threads)

- **Дедлайн:** 04.11.2026
- **Тема:** Введение в проект Loom (Virtual Threads)
- **Форма проверки:** задание с самопроверкой
- **Вариант итогового ДЗ:** 3 — парсер вакансий
- **Модуль:** `hw14.loom` в Maven-проекте `app/`
- **JDK:** 21 (Temurin 21.0.13), в `pom.xml` — `<java.version>21</java.version>`
- **Открытие задания на платформе:** 27.10.2026 в 17:20 (на момент подготовки текст задания недоступен, работа сделана по теме урока)

---

1. Что требовалось и что сделано

Тема урока — виртуальные потоки проекта Loom: зачем они нужны, чем отличаются от платформенных, где дают выигрыш. Текст задания на платформе ещё не открыт (появится 27.10.2026), поэтому сделано полное ДЗ по теме — с кодом, демонстрацией, тестами и измерениями.

Что	Где
Виртуальные потоки «базово»: загрузка N страниц вакансий	`LoomDemo` (этап 1), `benchmark.LoomBenchmarks.runNamedVirtual`
Сравнение с платформенным пулом	`LoomBenchmarks.runPlatform` / `runVirtual` (этап 2)
Pinning: `synchronized` против `ReentrantLock`	`pinning.PinningExperiment`, `pinning.PinningRun` (этап 3)
Демонстрация масштабирования (10 000 задач)	`LoomBenchmarks` + `benchmark.SleepyTask` (этап 4)
Локальный сайт вакансий (офлайн)	`server.VacancyPageServer`
Блокирующий HTTP-клиент + извлечение `<title>`	`web.PageVacancyClient`, `PageVacancyClient.HtmlTitle`
«Паспорт» потока: виртуальный и carrier	`benchmark.ThreadProbe`
Пик одновременной работы и число исполнителей	`benchmark.ConcurrencyMeter`
Итоги прогонов	`model.BenchmarkResult`, `model.LoadRun`, `model.PageLoad`
Строки отчёта и таблицы	`report.LoomReport`
Настройки `app.hw14.*`	`config.Hw14Properties`
Демонстрация	`LoomDemo`, запуск `--app.hw=hw14`

Предметная область та же, что в остальных ДЗ, — парсер вакансий. Страницы грузятся с локального сайта-заглушки на свободном порту, поэтому интернет не нужен, а задержка ответа делает разницу между исполнителями измеримой: виртуальные потоки масштабируются на **ожидании**, а не на процессоре.

Использованы именно те API, о которых говорит тема:

- `Executors.newVirtualThreadPerTaskExecutor()` — по виртуальному потоку на задачу;
- `Thread.ofVirtual().name("hw14-vt-", 0)` — именованные виртуальные потоки для журнала;
- `Thread.currentThread().isVirtual()` — проверка, что поток действительно виртуальный;
- `Thread.currentThread().toString()` — carrier-поток (`…/runnable@ForkJoinPool-1-worker-3`).

2. Этапы выполнения

Этап 1. Виртуальные потоки базово

```
 ThreadFactory factory = Thread.ofVirtual().name("hw14-vt-", 0).factory();
try (ExecutorService executor = Executors.newThreadPerTaskExecutor(factory)) {
    // на каждую страницу — своя задача; внутри блокирующий client.send(...)
}
```

`PageVacancyClient.fetch` намеренно **синхронный** (`HttpClient.send`, не `sendAsync`): именно блокирующий вызов позволяет виртуальному потоку сняться с carrier-потока и отдать его другим задачам. С асинхронным клиентом ждать было бы нечего и выигрыш исчез.

Каждый результат несёт «паспорт» потока (`ThreadProbe`): виртуальный ли он и на каком carrier смонтирован. Журнал демонстрации:

```
 виртуальные потоки: задач 200 (вернули результат 200, исключений 0, отменено 0), 190.4 мс, 10
                   пик одновременных 200, потоков-исполнителей 200, прирост потоков JVM 3
без потерь: да, разных страниц 200 из 200, сбоев 0
примеры заголовков:
  №0 «Вакансия №0 — Java-разработчик» (virtual@ForkJoinPool-1-worker-2)
  №1 «Вакансия №1 — Backend-разработчик (Java)» (virtual@ForkJoinPool-1-worker-4)
  №2 «Вакансия №2 — Разработчик Spring Boot» (virtual@ForkJoinPool-1-worker-6)
проверка: исполнений на виртуальных потоках 200, на платформенных 0 → все запросы сделаны вирт
паспорта потоков (первые 3):
  №0 поток hw14-vt-0, виртуальный: да, carrier: ForkJoinPool-1-worker-2
  №1 поток hw14-vt-1, виртуальный: да, carrier: ForkJoinPool-1-worker-4
  №2 поток hw14-vt-2, виртуальный: да, carrier: ForkJoinPool-1-worker-6
```

Наблюдения:

- 200 задач выполнились за 190 мс при задержке страницы 50 мс — то есть почти все ждали одновременно, а не по очереди;
- разные задачи попали на разные carrier-потоки (всего их 12 — по числу процессоров), при этом «потоков-исполнителей 200»: виртуальных потоков двести, потоков ОС — двенадцать;

- прирост счётчика платформенных потоков JVM — 3 (служебные потоки клиента), а не 200: виртуальные потоки в `ThreadMXBean` не входят;
- потерь нет: 200 разных страниц, 200 успешных загрузок, сбоев 0.

Перед замером выполняется прогрев на 32 страницах (в замеры не входит): первый HTTP-запрос подгружает классы клиента и искажает «холодное» число.

Этап 2. Тот же объём работы: платформенный пул и виртуальные потоки

Один и тот же список из 200 адресов, один и тот же клиент, один и тот же код задачи. Меняется только исполнитель: `Executors.newFixedThreadPool(16)` и `Executors.newVirtualThreadPerTaskExecutor()`.

```
 исполнитель         время, мс  стр./с  пик задач  потоков-исполнителей  исполнений virtual/pl
------------------  ---------  ------  ---------  --------------------  ----------------------
платформенный пул   720.9      277.4   16         16                    0/200
виртуальные потоки  128.9      1551.3  200        200                   200/0
вывод: тот же объём работы, виртуальные потоки быстрее в 5.6 раза (128.9 мс против 720.9 мс) –
       выигрыш даёт ожидание IO, а не процессор
```

Почему пул проигрывает ровно во столько раз, сколько потоков ему не хватает: 200 страниц × 50 мс / 16 потоков ≈ 625 мс — это нижняя граница для пула, и замеренные 721 мс к ней близки (остальное — накладные расходы). Виртуальные потоки не ждут очереди: все 200 запросов стартуют сразу, и время определяется одной задержкой (50 мс) плюс накладные расходы на 200 соединений.

Обратите внимание на две последние колонки: у пула 200 исполнений на платформенных потоках и +15–16 потоков ОС, у виртуальных — 200 исполнений на виртуальных потоках и **+0** потоков ОС. Это и есть ответ на вопрос «в чём разница»: количество работы то же, количество потоков ОС — нет.

Этап 3. Pinning: `synchronized` против `ReentrantLock`

Что такое pinning. Виртуальный поток снимается с carrier-потока на время блокировки — на этом и держится масштабирование. Но сняться можно не всегда: внутри `synchronized` поток удерживает монитор, и планировщик не может «заморозить» его на полпути. Такой поток **пришпиливается** (pinned) к carrier-потоку и блокирует его на всё время ожидания. `ReentrantLock` — очередь ожидания на уровне JDK, а не монитор JVM, поэтому с ним виртуальный поток снимается штатно.

Как поставлен опыт. У каждой задачи **свой** монитор (или свой `ReentrantLock`), поэтому состязания за общий ресурс нет — единственная разница между прогонами в том, пришпилен поток или нет. Внутри секции задача ждёт 60 мс. 96 задач, 12 carrier-потоков:

```
 защита        время, мс  пик задач  carrier'ов  смен carrier'а  ожидание при pinning, мс
-------------  ---------  ---------  ----------  --------------  ------------------------
synchronized   487.4      12         12 из 12    0               480.0
ReentrantLock  62.0       96         12 из 12    87              480.0
вывод: synchronized удерживает поток на carrier'е – одновременно работают не больше 12 задач,
       прогон идёт 487.4 мс; ReentrantLock отпускает carrier – одновременно работают все 96 за
       и прогон идёт 62.0 мс
```

Три независимых признака одного и того же явления:

1. **Пик одновременных задач.** При `synchronized` — 12 (по числу carrier-потоков), при `ReentrantLock` — 96 (все задачи).
2. **Время.** 96 задач по 60 мс в 12 «кругов» — это 8 × 60 = 480 мс; замеренные 487.4 мс совпадают с теорией. Свободные потоки проходят те же 96 задач за одну блокировку — 62.0 мс.
3. **Смена carrier'а.** Пришпиленный поток не мог сменить carrier — 0 смен. Свободный поток после сна просыпается на любом свободном carrier'е — 87 смен из 96 задач.

Это не «медленный synchronized»: сам монитор здесь не горячий, конкуренции нет. Разница только в том, что один механизм блокирует поток ОС, а второй — нет.

Этап 4. Масштабирование: 10 000 задач с ожиданием

```
 исполнитель             задач  время, мс  задач/с  пик задач  потоков-исполнителей  прирост по
---------------------  -------  ---------  -------  ---------  --------------------  -----------
платформенный пул 200  10000  2554.6     3914.5   200        200                   187
виртуальные потоки     10000  118.8      84147.3  10000      10000                 0
вывод: 10000 задач с ожиданием — виртуальные потоки быстрее в 21.5 раза; платформенный пул
       на 200 потоков ограничен их числом, виртуальные — нет
```

Расчёт для пула: 10 000 задач / 200 потоков = 50 порций × 50 мс = 2500 мс — совпадает с замеренными 2555 мс. Виртуальные потоки стартуют все сразу и ждут параллельно: 119 мс — это одна задержка плюс накладные расходы планировщика. При этом потоков ОС по-прежнему около двенадцати (прирост счётчика JVM — 0), а «пик задач» — 10 000: одновременно ждут десять тысяч виртуальных потоков, а не десять тысяч потоков ОС.

Именно поэтому в реальном парсере на 10 000 URL пул пришлось бы делать огромным (и он бы всё равно упёрся в память и переключения контекста), а виртуальные потоки дают тот же результат «из коробки».

3. Фактический вывод демонстрации

```
 cd app
java -jar target/vacancy-parser.jar --app.hw=hw14 --server.port=0
```

Прогон завершился с кодом `0`, живых платформенных потоков ДЗ 14 после завершения нет.

```
 === ДЗ 14. Проект Loom: виртуальные потоки на блокирующем IO ===


Процесс: pid 1430529
Настройки ДЗ 14:
    - страниц: 200, задержка страницы: 50 мс
    - платформенный пул для сравнения: 16 потоков
    - масштабирование: 10000 задач по 50 мс, пул 200
    - pinning: 96 задач по 60 мс
Планировщик виртуальных потоков: carrier-потоков (parallelism) 12, предел пула планировщика 25
```

```
Этап 1. Виртуальные потоки: загрузка страниц вакансий
    источник: локальный сайт, http://127.0.0.1:37119, страниц 200
    исполнитель: Thread.ofVirtual().name("hw14-vt-", 0) + newThreadPerTaskExecutor (по виртуал
    пример адреса: http://127.0.0.1:37119/vacancy/0
    прогрев: 32 страниц (в замеры не входят — первый запрос подгружает классы HTTP-клиента)
    виртуальные потоки: задач 200 (вернули результат 200, исключений 0, отменено 0), 190.4 мс,
    без потерь: да, разных страниц 200 из 200, сбоев 0
    ...
    проверка: исполнений на виртуальных потоках 200, на платформенных 0 → все запросы сделаны
    паспорта потоков (первые 3):
      №0 поток hw14-vt-0, виртуальный: да, carrier: ForkJoinPool-1-worker-2
      ...

Этап 2. Сравнение с платформенным пулом: тот же объём работы
    сначала newFixedThreadPool(16), затем Executors.newVirtualThreadPerTaskExecutor()
    исполнитель         время, мс  стр./с  пик задач  потоков-исполнителей  исполнений virtual
    ------------------  ---------  ------  ---------  --------------------  ------------------
    платформенный пул   720.9      277.4   16         16                    0/200
    виртуальные потоки  128.9      1551.3  200        200                   200/0
    вывод: тот же объём работы, виртуальные потоки быстрее в 5.6 раза (128.9 мс против 720.9 м
    целостность: платформенный пул — без потерь да, задач с результатом 200 из 200, страниц со
                 виртуальные потоки — без потерь да, задач с результатом 200 из 200, страниц с
    память (приблизительно, без System.gc()): пул 9.3 МиБ, виртуальные 10.5 МиБ

Этап 3. Pinning: synchronized против ReentrantLock
    у каждой задачи свой монитор/замок, внутри секции — ожидание 60 мс
    защита         время, мс  пик задач  carrier'ов  смен carrier'а  ожидание при pinning, мс
    -------------  ---------  ---------  ----------  --------------  ------------------------
    synchronized   487.4      12         12 из 12    0               480.0
    ReentrantLock  62.0       96         12 из 12    87              480.0
    вывод: synchronized удерживает поток на carrier'е — одновременно работают не больше 12 зад
    задачи завершены: synchronized 96 из 96, ReentrantLock 96 из 96
    как это читать: при pinning виртуальный поток не отдаёт carrier, поэтому одновременно ждут

Этап 4. Масштабирование: 10000 задач с ожиданием 50 мс
    исполнитель            задач  время, мс  задач/с  пик задач  потоков-исполнителей  прирост
    ---------------------  -----  ---------  -------  ---------  --------------------  -------
    платформенный пул 200  10000  2554.6     3914.5   200        200                   187
    виртуальные потоки     10000  118.8      84147.3  10000      10000                 0
    вывод: 10000 задач с ожиданием — виртуальные потоки быстрее в 21.5 раза; платформенный пул
    завершено задач: пул 10000 из 10000, виртуальные 10000 из 10000
    память (приблизительно): пул 0.0 МиБ (GC отработал во время прогона — замер приблизительны

Этап 5. Итог
    этап             исполнитель    время, мс  пик задач  итог
    ---------------  -------------  ---------  ---------  ----------------------------
    страницы         пул 16         720.9      16         все 200 страниц загружены
    страницы         виртуальные    128.9      200        все 200 страниц загружены
```

```
    pinning          synchronized   487.4      12         ограничен carrier'ами
    pinning          ReentrantLock  62.0       96         carrier освобождается
    масштабирование  пул 200        2554.6     200        задачи ждут в очереди пула
    масштабирование  виртуальные    118.8      10000      все задачи ждут одновременно

    когда виртуальные потоки полезны: тысячи одновременных блокирующих операций (HTTP, БД, фай
    когда нет: CPU-нагрузка (сортировка, сжатие, вычисления) — виртуальный поток всё равно зан
    чего избегать: synchronized вокруг блокирующих вызовов (pinning), пулы виртуальных потоков
    запросов к сайту за демонстрацию: 632, активных обработчиков сайта после этапов: 0

[tpm] Демонстрация завершена: все этапы пройдены, потерь задач нет, исполнители остановлены

живых платформенных потоков ДЗ 14 после завершения: []
виртуальные потоки в дамп не попадают: планировщик держит их вне Thread.getAllStackTraces(), г
```

`632` запроса — это 32 страницы прогрева плюс три полных прогона по 200 страниц (этап 1 и два прогона этапа 2).

4. Промежуточные результаты по этапам

Замеры на машине: 12 процессоров, 23 ГиБ ОЗУ, OpenJDK 21.0.13 (Temurin), Linux. Три последовательных прогона демонстрации:

Этап	Прогон 1	Прогон 2	Прогон 3	Вывод
200 страниц, виртуальные потоки	173.8 мс	157.0 мс	170.6 мс	≈1.2 стр./мс, все 200 одновременно
200 страниц, пул 16	726.0 мс	737.8 мс	734.2 мс	упирается в 16 потоков: 200 × 50 / 16 ≈ 625 мс
Ускорение (страницы, этап 2)	5.5×	5.7×	5.7×	стабильно в разы, не в проценты
Pinning: `synchronized`	486.6 мс, пик 12	486.9 мс, пик 12	486.3 мс, пик 12	8 кругов × 60 мс
Pinning: `ReentrantLock`	62.1 мс, пик 96	62.1 мс, пик 96	61.7 мс, пик 96	одна блокировка на всех
10 000 задач, пул 200	2542.4 мс	2549.6 мс	2544.3 мс	50 порций × 50 мс
10 000 задач, виртуальные	122.6 мс	123.3 мс	121.0 мс	одна задержка на всех

Качественная картина устойчива, абсолютные миллисекунды слегка плавают (±2 %) — это ожидаемо для wall-clock замера без JMH. Разброс не влияет на выводы: разница между исполнителями измеряется разами.

5. Когда виртуальные потоки полезны, а когда нет

Полезны:

- тысячи одновременных блокирующих операций: HTTP, JDBC, файлы, ожидание ответа внешнего сервиса;
- «поток на запрос» в сервере: простое последовательное программирование без реактивных цепочек;
- задачи с редкими, но длинными паузами, где пул пришлось бы делать очень большим.

Не полезны:

- CPU-нагрузка: виртуальный поток занимает carrier-поток на всё время вычислений, и 10 000 виртуальных потоков на 12 ядрах будут работать не быстрее 12 платформенных;
- внутри `synchronized` вокруг блокирующего вызова (pinning) — эффект исчезает;
- когда нужен ограниченный параллелизм: пул виртуальных потоков фиксированного размера бессмысленен, для ограничения числа одновременных операций нужен `Semaphore`, а не размер пула.

Чего избегать (pinning):

1. Не держать `synchronized` вокруг `Thread.sleep`, `InputStream.read`, `HttpClient.send`, `Future.get`, JDBC-вызовов. Заменить на `ReentrantLock` (или убрать блокировку из критической секции).
2. Не смешивать блокирующий IO и мониторы: критическая секция должна быть короткой и без ввода-вывода.
3. Следить за `jdk.virtualThreadScheduler.parallelism` и дампами: пришпиленные потоки видны в `jstack` как `VirtualThread[...] <pinned>` (в JDK 21; в JDK 24+ `synchronized` больше не пришпиливает — JEP 491).
4. Нативные вызовы (`JNI`) тоже пришпиливают поток — в JDK 21 это ограничение сохраняется.

6. Автоматические проверки

```
 cd app
mvn test -Dtest='ru.tsu.tpm.vacancyparser.hw14.**'
```

Результат: `Tests run: 19, Failures: 0, Errors: 0, Skipped: 0`.

Тест	Что проверяет
`ThreadProbeTest` (2)	виртуальный поток виден как виртуальный и сообщает carrier; у платформенного carrier'а нет
`ConcurrencyMeterTest` (2)	пик одновременной работы, уменьшение счётчика при выходе, раздельный учёт виртуальных и платформенных исполнений
`VacancyPageServerTest` (3)	детерминированные заголовки и адреса, реальное применение задержки, освобождение порта, некорректные параметры
`PageClientTest` (2)	разбор `<title>` (атрибуты, регистр, сущности, вложенные теги), недоступный сайт — результат-сбой, а не исключение
`LoomBenchmarksTest` (3)	все задачи виртуальные и с результатом; пул ограничен своими потоками; отмена по таймауту, сбой задачи, пустой прогон, некорректные параметры

Тест	Что проверяет
`PinningExperimentTest` (2)	`synchronized` ограничивает параллелизм числом carrier'ов и не даёт сменить carrier; `ReentrantLock` отпускает все задачи одновременно
`LoomReportTest` (2)	выравнивание таблиц, сравнения, примеры заголовков, итоговая таблица
`Hw14PropertiesBindingTest` (2)	значения по умолчанию и переопределение `app.hw14.*`
`LoomDemoConditionTest` (1)	демо создаётся только при `--app.hw=hw14`

Тесты проверяют **инварианты**, а не абсолютное время: миллисекунды зависят от машины, а «все задачи вернули результат», «работали виртуальные потоки», «при pinning пик не больше числа carrier'ов» должны выполняться всегда. Поэтому прогон не флакует.

Сборка целиком: `mvn -pl app package` — успешно. Полный прогон всех домашних заданий: `Tests run: 564, Failures: 0, Errors: 0, Skipped: 0` (было 545 + 19 новых).

7. Соответствие теме задания

Пункт темы	Где подтверждено	Отметка
Создание виртуальных потоков (`Thread.ofVirtual`, `newVirtualThreadPerTaskExecutor`)	этап 1, `LoomBenchmarks`	✅
Виртуальные потоки действительно виртуальные	`Thread.isVirtual()` в `ThreadProbe`, тесты	✅
Сравнение с платформенным пулом на IO	этап 2: 129 мс против 721 мс	✅
Пиковое число живых потоков и потребление	`ConcurrencyMeter`, `ThreadMXBean`, `MemoryProbe`	✅
Pinning: `synchronized` против `ReentrantLock`	этап 3: 487.4 мс / пик 12 против 62.0 мс / пик 96	✅
Масштабирование на 5 000–10 000 задачах	этап 4: 10 000 задач, 119 мс против 2555 мс	✅
Отчёт с этапами и фактическими числами	этот файл, разделы 2–4	✅

8. Как запустить и сдать

```
cd "Теория и практика многопоточности/app"

# Полная демонстрация (завершается с кодом 0)
mvn spring-boot:run -Dspring-boot.run.arguments="--app.hw=hw14 --server.port=0"

# Готовый jar
mvn -DskipTests package
java -jar target/vacancy-parser.jar --app.hw=hw14 --server.port=0
```

```
# Больше страниц и другой размер пула
java -jar target/vacancy-parser.jar --app.hw=hw14 --server.port=0 \
  --app.hw14.pages=400 --app.hw14.fixed-pool-size=8

# Меньше задач в масштабировании (быстрее, но контраст слабее)
java -jar target/vacancy-parser.jar --app.hw=hw14 --server.port=0 \
  --app.hw14.scaling-tasks=2000 --app.hw14.scaling-delay-millis=20
```

Параметры: `app.hw14.pages`, `app.hw14.page-delay-millis`, `app.hw14.fixed-pool-size`, `app.hw14.scaling-tasks`, `app.hw14.scaling-delay-millis`, `app.hw14.platform-pool-size`, `app.hw14.pinning-tasks`, `app.hw14.pinning-block-millis`, `app.hw14.request-timeout-millis`, `app.hw14.task-timeout-millis`, `app.hw14.sample-titles`.

Сдача: скопировать отчёт в Google/Яндекс Документ, приложить исходники или ссылку на репозиторий, открыть доступ и прикрепить ссылку в личном кабинете.

9. Ограничения и честные оговорки

- **Замер — wall-clock, не JMH.** Нет прогрева в сотни итераций и статистики по десяткам прогонов; числа воспроизводимы качественно (разы), но не до миллисекунды. Для учебной демонстрации этого достаточно, для выбора библиотеки — нет.
- **Память — оценка.** `Runtime.totalMemory() - freeMemory()` включает ещё не собранный мусор; GC может отработать между замерами, поэтому «прирост» бывает отрицательным (в отчёте это помечено). Точный замер требует `System.gc()` (искажает время) или JFR/async-profiler.
- **Счётчик платформенных потоков JVM общий на процесс и хранит исторический пик.** В него попадают и потоки Spring/Tomcat, и служебные потоки HTTP-клиента; если пик уже был достигнут раньше (например, в общем прогоне тестов), «прирост» равен нулю. Поэтому главный показатель — число потоков-исполнителей из `ConcurrencyMeter`, а счётчик JVM приведён как вспомогательная иллюстрация. Кроме того, локальный сайт сам обслуживает запросы виртуальными потоками: иначе его 200 обработчиков искажали бы замер клиента.
- **Виртуальные потоки не видны в `Thread.getAllStackTraces()` и обычном дампе** — проверить «не осталось ли живых» списком потоков нельзя, поэтому отсутствие потерь проверяется счётчиками задач (200 из 200, 10 000 из 10 000) и `executor.shutdown()`.
- **Задержка 50 мс и 200 страниц** — компромисс: меньше страниц — шумнее, больше — демонстрация удлиняется. 200 страниц × 16 потоков дают ~625 мс нижней границы для пула, что отличимо от ~130 мс у виртуальных потоков с запасом.
- **JDK 21.** На JDK 24+ (JEP 491) `synchronized` больше не пришпиливает виртуальные потоки, и этап 3 покажет другие числа: оба прогона пройдут за ~60 мс. Это не ошибка демонстрации, а изменение платформы; в отчёте указана версия JDK.
- **Текст задания на платформе ещё не открыт.** Если после 27.10.2026 требования окажутся другими (например, попросят конкретный класс, `StructuredTaskScope` или сравнение с `CompletableFuture`), модуль придётся дополнить — каркас (`LoomBenchmarks`, `ConcurrencyMeter`, `PinningExperiment`, тесты и отчёт) для этого уже есть.